

How to RICE Linux

Nicola Bruno

December 24, 2025



Contents

Contents	2
1 Introduction	4
1.1 What is this book	4
1.2 Who is this book aimed at	4
1.3 What is inside this book?	4
2 Let's get started	5
2.1 Before we begin	5
2.2 What makes a system	5
The Operating System	6
The display server	7
The window manager	8
The bar	8
The application launcher	9
The terminal emulator	9
The shell	9
The file manager	10
The notification daemon	10
The compositor	11
The wallpaper	11
The color scheme	12
The fonts	12
The audio system	12
The music player	13
System monitor	14
The lockscreen	14
The power menu	14
Screenshots	15
Clipboard manager	15
Configuration Files	15
Dotfile Managment	15
Miscellaneous cool stuff	15
Browser	16
Application Theming	16
Scripts and Automation	16
Login Manager	16
2.3 Going Forward	16

3	Base Configuration	17
3.1	The Operating System	17
	Debian	17
	Arch	17
	Fedora	18
3.2	Display Server	18
	X11	18
	Wayland	18
3.3	Window Manager	19
	X11 Compatible	19
	Wayland Compatible	19
3.4	Status bar	20
	X11 Compatible	20
	Wayland Compatible	20
	Compatible with both	21
3.5	Application launcher	21
	X11 Compatible	21
	Wayland Compatible	22
3.6	Terminal emulator	22
3.7	Shell	22
3.8	Compositors	23
	X11 Compatible	23
	Wayland Compatible	23
3.9	Wallpaper Managers	23
	X11 Compatible	23
	Wayland Compatible	23
3.10	Configuration	24
	X11 with i3wm	24
	Picom	26
	Wayland with Sway	27

3.10 Configuration

Now that we've chosen our pieces, we need to make them fit together. Here I will illustrate two example configurations: An X11 configuration with **i3wm**, and a Wayland configuration using **sway**. I will be using Arch linux as the OS on both setups, since is the most customizable OS that I can actually use. If you are using another distribution, just install the same packages with your package manager.

X11 with i3wm

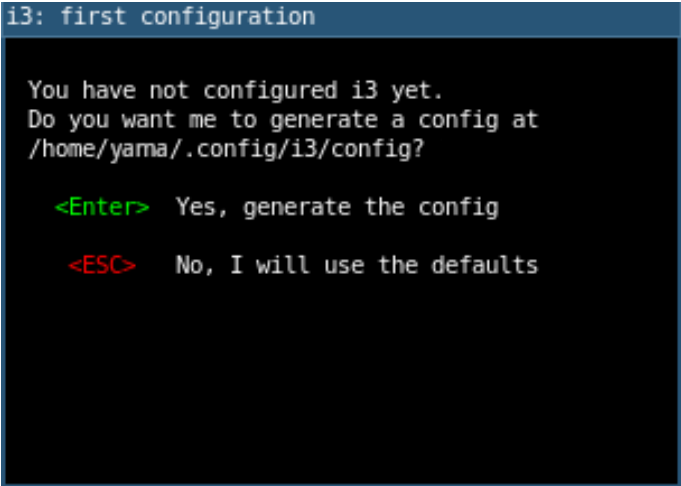
Our pieces will be:

- i3 as the window manager
- Picom as the compositor
- Polybar as the status bar
- Rofi as the application launcher
- Kitty as the terminal emulator
- Bash as the shell
- feh as the wallpaper manager

i3 config

If you're not familiar with tiling window managers, these are designed to have all of their behavior controlled from the keyboard with keybinds. I suggest having the **i3 Reference Card** near, so you can consult it when you don't know what to do.

The first thing we see when we boot into i3 for the first time is the default config generation prompt.

A terminal window with a dark background and a blue title bar that reads "i3: first configuration". The terminal text is as follows:

```
You have not configured i3 yet.  
Do you want me to generate a config at  
/home/yama/.config/i3/config?  
  
<Enter> Yes, generate the config  
<ESC> No, I will use the defaults
```

Figure 3.6: The default config generation prompt

After this, there is a similar prompt that makes us choose the **Mod** key, either the Windows key, or the Alt key. This can be changed later in the config file. The Mod key is a default key that is used to interact with the window manager's functions. For example, **Mod+Enter** opens a terminal window. This, as well as every other keybind, does not *require* the mod key, but is very useful to have all of our keybinds, both the default and the custom ones, using the Mod key.

Before we begin doing stuff, we need to set the resolution. To do this, we use `xrandr`. Just run this in the terminal and you should get the name of the connected displays, and the available resolutions. Then, just run:

```
xrandr --output <display-name> --mode <resolution>
```

If you do not have `xrandr`, install a package called **autorandr**, it autodefects resolution and refresh rate and sets them. If you want to change them, just run the command mentioned above

Kitty

Now that we know how to open the i3 terminal, we need to change the default terminal to ours of choice, in this case, **kitty**. To do this, we go inside the config file located in `/.config/i3/config` using our text editor, and change the line

```
#start a terminal
bindsym $mod+Return exec i3-sensible-terminal
```

to

```
#start a terminal
bindsym $mod+Return exec kitty
```

Now reload the environment with **Mod+Shift+c**. Now whenever you press the keybind for the terminal, it will open a Kitty instance.

After doing this, it's a good idea to change the keybinds to your liking, so that it's easier to change everything

Rofi

Now that we're comfortable with the keybinds, let's change the application launcher. The procedure is the same as the terminal, just change

```
#start dmenu (a program launcher)
bindsym $mod+d exec --no-startup-id dmenu
```

to

```
#start dmenu (a program launcher)
bindsym $mod+d exec --no-startup-id rofi -show drun
```

²Lines beginning with the `"#"` symbol are comments and do not affect the behavior of the config file. You can change `"start dmenu"` with `"start rofi"` if you desire so

Polybar

The next step is to change the awful looking i3 bar with Polybar. This is a little bit different, since the bar is started automatically and not with a keybind. To change the bar we need to go inside the **bar{}** function:

```
bar {  
    status_command i3-status  
}
```

And comment it out by adding **#** before each line. You can also delete it completely:

```
#bar {  
#    status_command i3-status  
#}
```

After doing this, we make the default bar polybar by adding this line somewhere inside the i3 config file:

```
exec_always --no-startup-id killall polybar || polybar
```

³ Now we just kill the bar with **killall i3bar** command inside a terminal.

Wallpaper

To set a wallpaper we need to configure **feh**, to do it for us. First of all, get a wallpaper from somewhere and store wherever you like. Now that we have the wallpaper, we just need to autostart feh whenever i3 boots up. We do that by adding this to the i3 config file:

```
exec --no-startup-id feh --bg-scale /path/to/wallpaper
```

Picom

Probably, while doing all of this stuff, you noticed some screen tearing, and the absolute lack of visual effects, like animations or transparency. That's because we did not install a compositor yet. Since we chose Picom, install it with your package manager. After installation, we need the config file, that probably does not exist, so to make sure, we're going to make it.

```
$ mkdir -p ~/.config/picom  
$ cp /etc/xdg/picom.conf ~/.config/picom/picom.conf
```

If that does not exist either, just make it inside the **.config/picom** directory and copy the default setting from the official documentation. Now that we have the configuration file ready to go, add it to the i3 config file:

```
exec --no-startup-id picom
```

```
exec --no-startup-id picom
```

³Here we add the **"||"** operator, which is OR, so that the shell tries to kill polybar, if it can't find it, it opens it. This is to make sure that every time we reload the session, we do not open another bar

And we're done!



Figure 3.7: The assembled i3 configuration

After a quick reboot to check that everything is working fine, we will have a basic i3 configuration that has custom keybinds, a custom application launcher, a custom terminal emulator, a custom bar, and a wallpaper. With a compositor to handle every effect that we can think of.

But of course, we are not really running custom stuff. This is just the scaffolding of our great desktop. We will be adding styling in the next chapter. So, if you have chosen X11 as your display server, feel free to skip the Wayland section ahead.

Wayland with Sway

Our pieces will be:

- Sway as the window manager
- Waybar as the status bar
- Rofi-wayland as the application launcher
- Kitty as the terminal emulator
- Bash as the shell
- swaybg as the wallpaper manager

Sway config

After installing Sway and booting for the first time, we may encounter two problems: The keyboard layout, and the resolution. Let's deal with the keys first. You will have to copy the config file from `/etc/sway/config` to `/.config/sway/config`. We do this with

```
$ cp /etc/sway/config ~/.config/sway/config
```

Now that we have our config file in a local directory, we change the input inside this file from:

```
input type:keyboard{
    xkb_layout "us"
}
```

To:

```
input type:keyboard{
    xkb_layout "your_layout"
}
```

Now that our keys work, let's fix the resolution with:

```
$ swaymsg get-outputs
```

This gives us a list of values. Just input your desired one in

```
$ swaymsg output<Monitor-Name> res <Desired-Resolution>
```

Now we have both working keys and the correct screen resolution.

Kitty

Let's set the terminal the same way we did on X11, by changing the keybind